

GoBall - System Design Document

Mini Golf Scoring System for Raspberry Pi 5

Ahmed Abdelghani

February 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Applicable Documents	3
1.4	Design Methodology	3
2	Design Constraints	4
2.1	Hardware Constraints	4
2.2	Software Constraints	4
2.3	Design Decisions	4
3	Data Design	5
3.1	Player Data Structure	5
3.2	LED Controller Data Structure	5
3.3	Color Constants	6
3.4	Flash Context	6
3.5	Game State Enumerations	6
3.6	Global Game State Variables	7
4	Module Detailed Design	8
4.1	GPIO and Game Logic Module (modules/logic/gpio_event.c)	8
4.1.1	Responsibilities	8
4.1.2	Initialization Sequence	8
4.1.3	Debounce Algorithm	8
4.1.4	Scoring Dispatch	8
4.1.5	Turn Management	9
4.2	Game Mode Modules (modules/game_modes/)	10
4.2.1	Stroke Play (strokeplay.c)	10
4.2.2	Match Play (matchplay.c + inline in gpio_event.c)	10
4.2.3	Quota Points (quotaplay.c)	10
4.2.4	Vegas Quota Points (quotaplay.c)	11
4.3	Sound Module (modules/sound_logic/)	12
4.3.1	Interface	12
4.3.2	Non-Blocking Playback Design	12
4.3.3	Sound File Organization	12
4.3.4	Playback Macros	12
4.3.5	Timing Configuration	13
4.4	LED Module (modules/led_logic/)	13
4.4.1	Initialization with Graceful Degradation	13

4.4.2	Train Animation Algorithm	13
4.4.3	Flash Effect Design	14
4.4.4	Brightness Scaling	14
4.5	Debug Module (<code>modules/debug/</code>)	14
4.5.1	Log Level Hierarchy	14
4.5.2	Compile-Time Elimination	15
4.5.3	Module Tagging	15
4.5.4	Features	15
4.6	UI Module (<code>modules/ui_logic/ + ui/</code>)	15
4.6.1	Two-Layer Event Architecture	15
4.6.2	Widget Naming Convention	16
4.6.3	Score Reset Design	16
4.6.4	Player Highlighting	16
5	Interface Design	17
5.1	Hardware Interfaces	17
5.1.1	GPIO Sensor Interface	17
5.1.2	PIO LED Interface	17
5.2	Software Interfaces	17
5.2.1	Module Interface Summary	17
6	Error Handling Design	19
6.1	Error Handling Strategy	19
6.2	Guard Patterns	19
7	Performance Design	20
7.1	Timing Budget	20
7.2	Memory Budget	20
7.3	Rendering Performance	20
8	Build Configuration Design	21
8.1	Cross-Compilation Toolchain	21
8.2	Debug Level Configuration	21
8.3	Source File Discovery	21
8.4	Link Dependencies	21
9	Appendix A: Sensor Pin Mapping	22
10	Appendix B: Sound File Inventory	22
10.1	Navigation Sounds	22
10.2	Mode Selection Sounds	22
10.3	Scoring Sounds	22
10.4	Turn Sounds	22
10.5	Winner Sounds (5 variants each)	23
11	Appendix C: LVGL Configuration Summary	23

1 Introduction

1.1 Purpose

This System Design Document (SDD) describes the detailed design of the GoBall mini golf scoring system. It covers data structures, algorithms, interface contracts, error handling strategies, and design decisions for each software module. This document complements the System Architecture Document (SAD) which provides the high-level structural overview.

1.2 Scope

GoBall is a Raspberry Pi 5-based scoring system that integrates:

- 4 IR sensors for ball detection across scoring holes (3, 4, 5, and 0 points)
- WS2812 LED strips for visual scoring feedback
- SDL2-based touchscreen UI built with LVGL and SquareLine Studio
- Voice announcements via SDL2_mixer
- Support for 5 game modes: Stroke Play, Match Play 1v1, Match Play 2v2, Quota Points, and Vegas Quota Points

1.3 Applicable Documents

Document	Description
System Architecture Document	High-level module structure and data flow
Software Requirements Specification	Functional and non-functional requirements
README.md	Build instructions and changelog

1.4 Design Methodology

The system follows an event-driven, single-threaded architecture with a cooperative main loop. Design priorities in order:

1. **Reliability** — hardware failures must not crash the system
2. **Responsiveness** — UI must remain fluid during sensor processing
3. **Simplicity** — flat module structure, minimal abstraction layers
4. **Testability** — graceful degradation enables desktop testing via QEMU

2 Design Constraints

2.1 Hardware Constraints

Constraint	Value	Impact
Target CPU	ARM Cortex-A76 (RPi5, aarch64)	Cross-compilation required
GPIO interface	/dev/gpiochip0 via libgpod	Kernel character device API
LED protocol	WS2812 at 800 kHz	Requires PIO hardware; unavailable on x86
Display	2560x720 touchscreen	Wide dual-panel layout for game + scorecard
Audio output	3.5mm analog or HDMI	SDL2_mixer with WAV files

2.2 Software Constraints

Constraint	Value	Impact
Language	C11	No C++ features; compatible with embedded toolchains
UI framework	LVGL v9.1.0	Screen definitions exported from SquareLine Studio
Display backend	SDL2	Single-window rendering with direct mode
Color depth	32-bit XRGB8888	Set in lv_conf.h as LV_COLOR_DEPTH = 32
Memory budget	512 MB LVGL heap	LV_MEM_SIZE = 512 * 1024 * 1024
Refresh rate	62.5 Hz	LV_DEF_REFR_PERIOD = 16 ms
Build system	CMake >= 3.15	Cross-toolchain via toolchain-aarch64.cmake

2.3 Design Decisions

Decision	Rationale
Single-threaded main loop	Avoids mutex complexity; LVGL is not thread-safe
Inline scoring in gpio_event.c	All game state mutations in one file prevents race conditions
POSIX signal-based debounce	More reliable than timestamp comparison; per-sensor independence
WAV files (not MP3/OGG)	Zero decoding latency; SDL2_mixer plays directly
PIO for LEDs (not bit-bang)	Precise 800 kHz timing without CPU blocking
Static player array	Player players[4] avoids heap allocation for game state

3 Data Design

3.1 Player Data Structure

```
typedef struct Player {
    int      score;           // Cumulative score (Stroke Play, Quota)
    int      current_hole;    // Zero-based hole index (0..8 or 0..17)
    int      detection_count; // Detections in current turn (0..2)
    int      holes_won;       // Match Play: holes won count
    char     match_status[10]; // Match Play: status string
    int      holes_halved;    // Match Play: halved holes count
    signed char upAndDown;    // Match Play: differential (-N..+N)
    int      round_total_score; // Per-round cumulative (Match Play)
    int      par3_count;      // Quota: remaining 3pt pars
    int      par4_count;      // Quota: remaining 4pt pars
    int      par5_count;      // Quota: remaining 5pt pars
} Player;
```

Storage: Static array `Player players[MAX_PLAYERS]` where `MAX_PLAYERS = 4`.

Lifecycle:

1. Zeroed on `logic_initialize_game()` call
2. Par counts set by UI callbacks when entering Quota/Vegas game screens
3. Modified by scoring functions during gameplay
4. Reset by `reset_scores()` on game exit or new game

Field usage by game mode:

Field	Stroke Play	Match Play	Quota	Vegas
<code>score</code>	Cumulative	–	Bonus score	Bonus score
<code>current_hole</code>	Yes	Yes	Yes	Yes
<code>detection_count</code>	Yes	Yes	Yes	Yes
<code>holes_won</code>	–	Yes	–	–
<code>upAndDown</code>	–	Yes	–	–
<code>round_total_score</code>	–	Per-hole	Per-hole	Per-hole
<code>par3_count</code>	–	–	Countdown	Countdown
<code>par4_count</code>	–	–	Countdown	Countdown
<code>par5_count</code>	–	–	Countdown	Countdown

3.2 LED Controller Data Structure

```
#define NUM_TRAINS    5
#define TRAIN_LENGTH  10
#define PIXELS        144
```

```
typedef struct {
    PIO      pio;           // PIO hardware handle
    uint8_t  sm1, sm2;      // State machine IDs
    uint8_t  offset;        // Program offset in PIO memory
    uint8_t  gpio1, gpio2;  // Data pins (default: 3, 2)
    uint8_t  train_positions[NUM_TRAINS]; // Animation positions
    uint8_t  databuf1[PIXELS * 4]; // Strip 1: 576 bytes (WBGR)
    uint8_t  databuf2[PIXELS * 4]; // Strip 2: 576 bytes (WBGR)
    uint8_t  brightness;    // Global brightness (0-255)
```

```

    bool    enabled;                                // Hardware availability flag
} led_strip_controller_t;

```

Memory layout per pixel (4 bytes):

Byte offset	Channel	Bit position
4*i + 0	White	MSB
4*i + 1	Blue	
4*i + 2	Green	
4*i + 3	Red	LSB

Global instance: `led_strip_controller_t leds` (single global, not heap-allocated).

3.3 Color Constants

```

typedef struct {
    uint8_t w, b, g, r;
} wbgr_color_t;

const wbgr_color_t COLOR_RED   = {0x00, 0x00, 0x00, 0xFF};
const wbgr_color_t COLOR_GREEN = {0x00, 0x00, 0xFF, 0x00};
const wbgr_color_t COLOR_BLUE  = {0x00, 0xFF, 0x00, 0x00};
const wbgr_color_t COLOR_WHITE = {0xFF, 0xFF, 0xFF, 0xFF};

```

3.4 Flash Context

```

typedef struct {
    led_strip_controller_t* controller;
    wbgr_color_t            color;
} flash_context_t;

```

Heap-allocated via `malloc()` in `trigger_flash_with_color()`. Ownership passes to the LVGL timer system; freed implicitly when the timer completes.

3.5 Game State Enumerations

```

typedef enum {
    GAME_MODE_STROKE_PLAY,    // 0
    GAME_MODE_MATCH_PLAY,     // 1
    GAME_MODE_QUOTA,          // 2
    GAME_MODE_VEGAS,          // 3
    NUM_GAME_MODES             // 4 (sentinel)
} GameMode;

#define NINE_HOLES      8    // Zero-based max index for 9 holes
#define EIGHTEEN_HOLES 17    // Zero-based max index for 18 holes

typedef enum {
    HOLES_9  = NINE_HOLES,    // 8
    HOLES_18 = EIGHTEEN_HOLES // 17
} HoleMode;

typedef enum {
    MATCH_PLAY_MODE_1V1,      // 0

```

```

    MATCH_PLAY_MODE_2V2    // 1
} MatchPlayMode;

```

Design note: NINE_HOLES = 8 and EIGHTEEN_HOLES = 17 are zero-based maximum indices, not hole counts. A player has completed all holes when `current_hole > NINE_HOLES` (i.e., `current_hole >= 9`).

3.6 Global Game State Variables

Variable	Type	Defined In	Purpose
<code>current_game_mode</code>	<code>GameMode</code>	<code>gpio_event.c</code>	Active game mode
<code>current_hole_mode</code>	<code>HoleMode</code>	<code>gpio_event.c</code>	9 or 18 holes
<code>current_match_play_mode</code>	<code>MatchPlayMode</code>	<code>gpio_event.c</code>	1v1 or 2v2
<code>players[MAX_PLAYERS]</code>	<code>Player[4]</code>	<code>gpio_event.c</code>	Player state
<code>current_player_index</code>	<code>int</code>	<code>gpio_event.c</code>	Active player (0-based)
<code>num_players</code>	<code>int</code>	<code>gpio_event.c</code>	Active player count
<code>sensors_enabled</code>	<code>bool</code>	<code>gpio_event.c</code>	Sensor input gate
<code>update_flag</code>	<code>uint8_t</code>	<code>gpio_event.c</code>	Game completion flag
<code>sensor_debouncing[4]</code>	<code>bool[4]</code>	<code>gpio_event.c</code>	Per-sensor debounce state
<code>leds</code>	<code>led_strip_controller_t</code>	<code>led_logic_event.c</code>	LED controller instance

4 Module Detailed Design

4.1 GPIO and Game Logic Module (`modules/logic/gpio_event.c`)

4.1.1 Responsibilities

- GPIO hardware initialization via libgpiod
- Sensor event polling (non-blocking)
- Debounce management (POSIX signal timers)
- Scoring dispatch to game mode handlers
- Turn management and player rotation
- Scorecard UI updates
- Game completion detection

4.1.2 Initialization Sequence

```
logic_initialize_game(num_players)
|
+-> Validate num_players (1..MAX_PLAYERS)
+-> Zero-initialize players[0..num_players-1]
+-> init_debounce_timers()
|   +-> Create 4 POSIX timers (one per sensor)
|   +-> Register SIGRTMIN signal handler
|   +-> Handler clears sensor_debouncing[index] on expiry
+-> logic_gpio_init()
    +-> gpiod_chip_open("/dev/gpiochip0")
    +-> gpiod_chip_get_lines(pins[17,26,27,24])
    +-> gpiod_line_request_bulk_falling_edge_events()
```

Error handling: Each step returns -1 on failure. `main.c` catches failure and sets `gpio_available = 0`, enabling UI-only mode.

4.1.3 Debounce Algorithm

```
Sensor Event Arrives (GPIO falling edge)
|
+-> sensor_debouncing[sensor_index] == true?
|   YES -> Ignore event (timer still active)
|   NO  -> Continue processing
|
+-> sensor_debouncing[sensor_index] = true
+-> arm_timer(sensor_index, DEBOUNCE_TIME_MS=3000)
+-> Process scoring event
|
... 3 seconds later ...
|
SIGRTMIN fires -> debounce_signal_handler()
+-> sensor_debouncing[sensor_index] = false
+-> Sensor ready for next event
```

Design rationale: Signal-based timers are more reliable than `clock_gettime()` comparison because they handle system clock adjustments and don't drift under CPU load. Each sensor has an independent timer, allowing simultaneous detection on different holes.

4.1.4 Scoring Dispatch

The scoring chain processes a validated, debounced pin event:

```

// Simplified flow (actual code is inline in logic_handle_events)
Player* player = &players[current_player_index];

if (player->detection_count < SENSORS_PER_TURN) {
    switch (current_game_mode) {
        case GAME_MODE_STROKE_PLAY:
            stroke_play_process_pin(player, index, pin, &leds);
            break;
        case GAME_MODE_MATCH_PLAY:
            // Inline: player->round_total_score += score
            break;
        case GAME_MODE_QUOTA:
            quota_play_process_pin(player, index, pin, &leds);
            player->round_total_score = player->score;
            break;
        case GAME_MODE_VEGAS:
            vegas_quota_play_process_pin(player, index, pin, &leds,
                                         players, num_players);
            player->round_total_score = player->score;
            break;
    }
    player->detection_count++;
    logic_update_label_text(...);
    // ... completion checks, turn advancement
}

```

4.1.5 Turn Management

Turn advancement follows this pattern:

Condition	Action
detection_count < SENSORS_PER_TURN	Continue current turn
detection_count == SENSORS_PER_TURN	Complete turn, advance player
Quota/Vegas: par3==0 && par4==0 && par5==0	Immediate game end
Quota/Vegas 1P: after 1 detection	Reset count (no 2-ball requirement)

Player rotation:

```
current_player_index = (current_player_index + 1) % num_players;
```

Turn cycling patterns for player highlights:

Players	Detections per cycle	Pattern
2P	8 (4 turns x 2 balls)	P1, P1, P2, P2, P1, P1, P2, P2
3P	12 (6 turns x 2 balls)	P1, P1, P2, P2, P3, P3, repeat
4P	24 (12 turns x 2 balls)	P1, P1, P2, P2, P3, P3, P4, P4, repeat

4.2 Game Mode Modules (`modules/game_modes/`)

4.2.1 Stroke Play (`strokeplay.c`)

Interface:

```
void stroke_play_process_pin(Player* player, int player_index,
                           unsigned int gpio_pin,
                           led_strip_controller_t* leds);
```

Algorithm:

1. Null-check `player` (error) and `leds` (warn, continue)
2. Switch on `gpio_pin`:
 - Pin 17: `score += 3`, green flash, “3 points” sound
 - Pin 26: `score += 4`, green flash, “4 points” sound
 - Pin 27: `score += 5`, green flash, “5 points” sound
 - Pin 24: `score += 0`, white flash, “0 points” sound
3. Score is cumulative across all holes

Winner determination: After all holes completed, `check_all_players_completed()` compares `players[i].score` values. Highest score wins.

4.2.2 Match Play (`matchplay.c` + inline in `gpio_event.c`)

Scoring algorithm (per hole):

Both players play the hole (2 detections each)

```
|
+--> Compare round_total_score
|   P1 > P2: players[0].upAndDown++, players[1].upAndDown--
|   P2 > P1: players[0].upAndDown--, players[1].upAndDown++
|   Equal:  no change (halved)
|
+--> Reset round_total_score for both
+--> Advance both to next hole
```

Early victory condition:

```
int lead = max(players[0].upAndDown, players[1].upAndDown);
int holes_remaining = current_hole_mode - players[0].current_hole - 1;
bool early_victory = (lead > 0 && lead > holes_remaining);
```

If `early_victory == true`, the game ends immediately without playing remaining holes. The winner display uses “X&Y” format (e.g., “3&2” means 3 up with 2 holes remaining).

UI display format:

State	Player 1	Player 2
P1 leads by 2	“2UP”	“2DN”
P2 leads by 1	“1DN”	“1UP”
Even	“E”	“E”

4.2.3 Quota Points (`quotaplay.c`)

Interface:

```
void quota_play_process_pin(Player* player, int player_index,
                           unsigned int gpio_pin,
                           led_strip_controller_t* leds);
```

Algorithm:

1. Switch on gpio_pin:
 - Pin 17: score += 3, decrement par3_count if > 0
 - Pin 26: score += 4, decrement par4_count if > 0
 - Pin 27: score += 5, decrement par5_count if > 0
 - Pin 24: no score change, no par change
2. LED flash and sound for all holes

Completion test:

```
int quota_player_completed(Player* player) {
    return (player->par3_count == 0 &&
            player->par4_count == 0 &&
            player->par5_count == 0);
}
```

4.2.4 Vegas Quota Points (quotaplay.c)**Interface:**

```
void vegas_quota_play_process_pin(Player* player, int player_index,
                                  unsigned int gpio_pin,
                                  led_strip_controller_t* leds,
                                  Player* all_players,
                                  uint8_t num_players);
```

Bonus scoring algorithm:

For each category (3pt, 4pt, 5pt):

- Count how many players have closed it (par_count == 0)
- Category is "alive" if:
 - Single player mode (always alive), OR
 - Fewer than 2 players have closed it

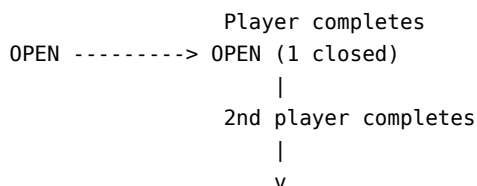
On ball detection:

- If par_count > 0:
 - Decrement par_count (countdown phase)
- Else if category alive:
 - Add bonus points to score
- Else:
 - No bonus (category dead)

Dead category rule (multiplayer):

```
int closed_par3 = 0;
for (uint8_t i = 0; i < num_players; i++)
    if (all_players[i].par3_count == 0) closed_par3++;

int par3_alive = (num_players == 1) || (closed_par3 < 2);
```

State diagram per category:

DEAD (no bonus for anyone)

4.3 Sound Module (`modules/sound_logic/`)

4.3.1 Interface

```
int      init_audio_system();           // SDL2_mixer init
void     cleanup_audio_system();        // Resource cleanup
Mix_Chunk* load_sound_effect(const char* path); // WAV loading
void     play_sound_effect(Mix_Chunk* sound); // Immediate play
lv_timer_t* play_sound_once(Mix_Chunk* sound, uint32_t delay_ms); // Delayed
void     stop_all_sounds();
void     mute_audio();
void     unmute_audio();
int      is_audio_muted();
```

4.3.2 Non-Blocking Playback Design

Sound playback must not block the 5ms main loop. The design uses LVGL's timer system:

```
play_sound_once(sound, delay_ms)
|
+-> Create lv_timer with delay_ms period
+-> Set repeat count = 1 (one-shot)
+-> Store Mix_Chunk* as timer user_data
+-> Return immediately
|
... delay_ms later (during lv_timer_handler) ...
|
+-> Timer callback fires
+-> Mix_PlayChannel(-1, sound, 0)
+-> Timer self-deletes
```

4.3.3 Sound File Organization

All WAV files reside in `modules/game_sounds/`. The base path is configurable:

```
#ifndef SOUND_DIR
#define SOUND_DIR "modules/game_sounds/" // Development default
#endif
// Override for Yocto: cmake -DSOUND_DIR_PATH="/usr/share/goball/sounds/"
```

4.3.4 Playback Macros

Two patterns are used:

Immediate (navigation sounds):

```
#define PLAY_BACKBTN_SOUND \
    if (!is_audio_muted()) \
        play_sound_effect(load_sound_effect(SOUND_BACKBTN_WAV));
```

Randomized with delay (winner announcements):

```
#define PLAY_PLAYER1WINS_WAV \
do { \
    const char* sounds[] = { \
        SOUND_PLAYER1WINS1_WAV, \
```

```

        SOUND_PLAYER1WINS2_WAV,          \
        SOUND_PLAYER1WINS3_WAV,          \
        SOUND_PLAYER1WINS4_WAV,          \
        SOUND_PLAYER1WINS5_WAV           \
    };                                    \
    int idx = rand() % 5;                  \
    Mix_Chunk* s = load_sound_effect(sounds[idx]);\
    if (s) play_sound_once(s, SOUND_DELAY_PLAYER_WINS_MS); \
} while (0)

```

4.3.5 Timing Configuration

Constant	Default	Purpose
SOUND_DELAY_PLAYER_ANNOUNCE_MS	750 ms	“Player X” turn announcement
SOUND_DELAY_TEAM_ANNOUNCE_MS	750 ms	“Team X” turn announcement
SOUND_DELAY_PLAYER_WINS_MS	750 ms	“Player X wins!” announcement
SOUND_DELAY_TEAM_WINS_MS	750 ms	“Team X wins!” announcement
SOUND_DELAY_TURN_SWITCH_MS	500 ms	Next player/team announcement

4.4 LED Module (`modules/led_logic/`)

4.4.1 Initialization with Graceful Degradation

```

init_led_controller(gpio1=3, gpio2=2)
|
+--> Set defaults (brightness=10)
+--> pio_open(0)
|   FAIL (PIO_IS_ERR) -> enabled=false, return
|   OK -> continue
+--> pio_claim_unused_sm() x2
|   FAIL (sm < 0) -> enabled=false, return
|   OK -> continue
+--> pio_sm_config_xfer() x2
+--> pio_add_program(&ws2812_program)
+--> pio_sm_clear_fifos() x2
+--> pio_sm_set_clkdiv(1.0) x2
+--> ws2812_program_init() x2 (800kHz, non-RGBW)
+--> initialize_train_positions()
+--> enabled = true

```

Key design point: Uses `pio_open()` + `PIO_IS_ERR()` instead of the `pio0` macro (which calls `pio_open_helper()` that `exit(1)` on failure). This allows the application to continue without LEDs on non-RPi5 hardware.

4.4.2 Train Animation Algorithm

The idle animation displays colored “trains” traveling along the LED strip:

```

void update_led_animation(led_strip_controller_t* controller) {
    if (!controller->enabled || animation_paused) return;

    // 1. Fill both strips with background (white, brightness-scaled)
    update_led_strip(controller, databuf1, train_positions, ...);
    update_led_strip(controller, databuf2, train_positions, ...);
}

```

```

// 2. Push data to hardware
pio_sm_xfer_data(pio, sm1, databuf1);
pio_sm_xfer_data(pio, sm2, databuf2);

// 3. Advance all train positions
for (t = 0; t < NUM_TRAINS; t++)
    train_positions[t] = (train_positions[t] + 1) % PIXELS;
}

```

Train initialization: Trains are evenly spaced:

```

train_positions[i] = (i * PIXELS) / NUM_TRAINS;
// For PIXELS=144, NUM_TRAINS=5: positions = {0, 28, 57, 86, 115}

```

Per-pixel rendering in update_led_strip():

1. Set all pixels to white (background) at current brightness
2. For each train, overwrite TRAIN_LENGTH=10 consecutive pixels with train color (dark green: W=0x00, B=0x00, R=0x08, G=0x65)

4.4.3 Flash Effect Design

```

trigger_flash_with_color(controller, duration_ms, color)
|
+-> animation_paused = true
+-> clear_all_leds()
+-> malloc(flash_context_t) with color
+-> Create flash_timer (50ms interval, duration/250 cycles)
|
+-> flash_toggle(): alternates between color and off
+-> Create restore_timer (duration_ms, one-shot)
+-> restore_animation(): animation_paused = false

```

Flash timing example (1000ms green flash for scoring):

- Flash timer: 50ms interval, 4 cycles = 200ms of toggling
- Restore timer: 1000ms, resumes animation
- Visual: rapid on/off blinks, then animation resumes

4.4.4 Brightness Scaling

```

uint8_t scale_brightness(uint8_t color, uint8_t brightness) {
    return (color * brightness) / 255;
}

```

Linear scaling. At brightness=50 (default), a full-intensity channel (0xFF) outputs $(255 * 50) / 255 = 50$.

4.5 Debug Module (modules/debug/)

4.5.1 Log Level Hierarchy

```

typedef enum {
    DEBUG_LEVEL_NONE   = 0, // No output
    DEBUG_LEVEL_ERROR  = 1, // Unrecoverable errors
    DEBUG_LEVEL_WARN   = 2, // Handled anomalies
    DEBUG_LEVEL_INFO   = 3, // Operational milestones
    DEBUG_LEVEL_DEBUG  = 4, // Diagnostic detail
    DEBUG_LEVEL_TRACE  = 5  // Per-frame, function entry/exit
} debug_level_t;

```

4.5.2 Compile-Time Elimination

Each log macro is wrapped in a preprocessor guard:

```
#if DEBUG_LEVEL >= DEBUG_LEVEL_INFO
#define DEBUG_INFO(module, format, ...) \
    DEBUG_PRINT(DEBUG_LEVEL_INFO, DEBUG_MODULE_NAMES[module], \
                format, ##__VA_ARGS__)
#else
#define DEBUG_INFO(module, format, ...) ((void)0)
#endif
```

In Release builds (DEBUG_LEVEL=2), DEBUG_INFO, DEBUG_DEBUG, and DEBUG_TRACE compile to no-ops with zero runtime cost.

4.5.3 Module Tagging

```
typedef enum {
    MODULE_MAIN, MODULE_LVLG, MODULE_UI, MODULE_GAME,
    MODULE_SOUND, MODULE_LED, MODULE_GPIO, MODULE_INPUT,
    MODULE_ANIMATION, MODULE_LOGIC, MODULE_HAL,
    MODULE_COUNT
} debug_module_t;
```

Each log line includes the module name for filtering:

```
[INFO ] [GAME ] Player 1 scored 3 points
[DEBUG] [LED  ] Flash timer created: interval=50ms, cycles=4
[WARN ] [GPIO ] PIO hardware not available - LEDs disabled
```

4.5.4 Features

- **ANSI colors:** Error=red, Warn=yellow, Info=green, Debug=cyan, Trace=magenta (toggle: ENABLE_DEBUG_COLORS)
- **Timestamps:** Millisecond precision via clock_gettime(CLOCK_MONOTONIC) (toggle: ENABLE_DEBUG_TIMESTAMP)
- **File and line:** Each log includes __FILE__:__LINE__ for source tracing

4.6 UI Module (modules/ui_logic/ + ui/)

4.6.1 Two-Layer Event Architecture

Layer 1: SquareLine Studio generated (ui/ui.c)

- Screen definitions and widget tree construction
- Button event callbacks for navigation (_ui_screen_change())
- Sound playback macros on button press
- Approximately 7400 lines, auto-generated

Layer 2: Custom logic (modules/ui_logic/ui_logic.event.c)

- Game mode setup from UI selections (player count, hole count)
- Live score display updates
- Player highlighting (background color changes)
- Crown icon visibility for winners
- Scorecard population
- Screen transitions on game completion
- Approximately 6500 lines

4.6.2 Widget Naming Convention

SquareLine Studio generates widget names with encoded metadata:

```
ui_[Mode][Players][Holes][Screen][Widget][Detail]
```

Examples:

```
ui_SP2P9HGSBCPText    Stroke Play, 2P, 9H, Game Screen, Ball Counter
ui_MP1V19HScP1SText3  Match Play, 1v1, 9H, Scorecard, P1, Hole 3
ui_VQ3P18HGSBSText    Vegas Quota, 3P, 18H, Game Screen, Bonus Score
ui_Q4P9HGSMMButton    Quota, 4P, 9H, Game Screen, Main Menu Button
```

4.6.3 Score Reset Design

`reset_scores()` resets all UI widgets to initial state:

- Stroke Play scorecards: all cells to “0”
- Match Play scorecards: all cells to “-”
- Ball counters: reset to “2”
- Crown icons: hidden (`lv_obj_add_flag(LV_OBJ_FLAG_HIDDEN)`)
- Bonus score labels (Vegas): reset to “0”
- Player highlights: reset to default colors

4.6.4 Player Highlighting

Background colors cycle based on detection count:

Color	Hex	Usage
Active (bright)	0x34bb19	Current player’s turn
Active (dim)	0x5AB66A	Secondary highlight
Inactive	Default	Not current player’s turn

5 Interface Design

5.1 Hardware Interfaces

5.1.1 GPIO Sensor Interface

```
// Pin mapping
unsigned int sensor_pins[NUM_SENSORS] = {17, 26, 27, 24};

// Initialization
struct gpiod_chip* chip; // /dev/gpiochip0
struct gpiod_line_bulk lines; // Bulk line handle

// Event detection
gpiod_line_event_wait_bulk(&lines, &(struct timespec){0, 0}, &event_lines);
// Non-blocking: timeout = {0, 0}
// Returns: >0 if events pending, 0 if none, <0 on error
```

5.1.2 PIO LED Interface

```
// Initialization
PIO pio = pio_open(0); // Open PIO device
int sm = pio_claim_unused_sm(pio, false); // Claim state machine
int off = pio_add_program(pio, &ws2812_program); // Load WS2812 program

// Data transfer (per frame)
pio_sm_xfer_data(pio, sm, PIO_DIR_TO_SM, sizeof(databuf), databuf);
// Sends PIXELS*4 bytes to state machine for serial output
```

5.2 Software Interfaces

5.2.1 Module Interface Summary

Function	Module	Called By	Purpose
logic_initialize_game()	gpio_event	main	Setup game state + GPIO
logic_handle_events()	gpio_event	main	Process sensor events
init_audio_system()	sound_logic	main	SDL2_mixer init
init_led_controller()	led_logic	main	PIO + LED init
update_led_animation()	led_logic	main	Per-frame LED update
set_game_mode()	gpio_event	ui_logic	UI game mode selection
set_hole_mode()	gpio_event	ui_logic	UI hole count selection
set_num_players()	gpio_event	ui_logic	UI player count selection
reset_scores()	gpio_event	ui_logic	Reset all game state + UI
stroke_play_process_pin()	strokeplay	gpio_event	Score a stroke play event
quota_play_process_pin()	quotaplay	gpio_event	Score a quota event
vegas_quota_play_process_pin()	quotaplay	gpio_event	Score a vegas event
trigger_flash_with_color()	led_logic	game modes	LED flash effect
play_sound_once()	sound_logic	gpio_event	Delayed sound playback
check_all_players_completed()	gpio_event	gpio_event	End-of-game detection

Function	Module	Called By	Purpose
----------	--------	-----------	---------

6 Error Handling Design

6.1 Error Handling Strategy

The system uses a **fail-soft** approach: hardware failures disable the affected subsystem while allowing the rest of the application to continue.

Subsystem	Failure Mode	Handling	User Impact
GPIO	/dev/gpiochip0 not found	<code>gpio_available = 0</code>	UI-only mode (no scoring)
PIO	PIO device unavailable	<code>leds.enabled = false</code>	No LED feedback
PIO	State machines unavailable	<code>leds.enabled = false</code>	No LED feedback
Audio	SDL2_mixer init fails	Warning logged	No sound (game still works)
Sound file	WAV not found	<code>load_sound_effect()</code> returns NULL	Silent (no crash)
Debounce timer	Timer creation fails	<code>logic_initialize_game()</code> returns -1	Falls back to UI-only

6.2 Guard Patterns

LED guard (every LED function):

```
void set_all_leds(led_strip_controller_t* controller, uint32_t color) {
    if (!controller->enabled) return; // Early exit if no hardware
    // ... proceed with PIO operations
}
```

Sensor guard (main loop):

```
if (gpio_available) {
    logic_handle_events(&event_lines, &event, num_players);
}
```

Sound guard (all playback macros):

```
if (!is_audio_muted())
    play_sound_effect(load_sound_effect(SOUND_PATH));
// load_sound_effect returns NULL on missing file
// play_sound_effect handles NULL gracefully
```

Null parameter guards (game mode functions):

```
if (player == NULL) {
    DEBUG_ERROR(MODULE_GAME, "Invalid player pointer (NULL)");
    return;
}
if (leds == NULL) {
    DEBUG_WARN(MODULE_GAME, "LED controller is NULL - visual feedback disabled");
    // Continue without LED effects
}
```

7 Performance Design

7.1 Timing Budget

The main loop targets a 5ms cycle (200 Hz):

Step	Operation	Typical Time	Blocking?
1	<code>update_led_animation()</code>	< 0.5ms	No (PIO DMA)
2	<code>logic_handle_events()</code>	< 0.1ms	No (non-blocking poll)
3	<code>lv_timer_handler()</code>	1-4ms	Partial (rendering)
4	<code>usleep(5000)</code>	5ms	Yes (intentional)
Total		~6-10ms	

7.2 Memory Budget

Component	Allocation	Size
LVGL heap	<code>LV_MEM_SIZE</code>	512 MB
LVGL image cache	<code>LV_CACHE_DEF_SIZE</code>	8 MB
LVGL draw buffer	<code>LV_DRAW_LAYER_SIMPLE_BUF_SIZE</code>	16 MB
LED data buffers	<code>PIXELS * 4 * 2</code>	1,152 bytes
Player state	<code>sizeof(Player) * 4</code>	~200 bytes
Flash context	<code>malloc</code> per flash	~12 bytes (freed after)
Sound chunks	SDL2_mixer managed	Variable (WAV PCM data)

7.3 Rendering Performance

Setting	Value	Impact
Draw units	4 parallel (<code>LV_DRAW_SW_DRAW_UNIT_CNT</code>)	Multi-core rendering
Render mode	Direct (<code>LV_SDL_RENDER_MODE_DIRECT</code>)	Zero-copy to SDL
Buffer count	1 (<code>LV_SDL_BUF_COUNT</code>)	Single buffer, no tearing issues at 62.5 Hz
DPI	130 (<code>LV_DPI_DEF</code>)	Touch accuracy calibration

8 Build Configuration Design

8.1 Cross-Compilation Toolchain

toolchain-aarch64.cmake

```
|
+--> CMAKE_SYSTEM_NAME = Linux
+--> CMAKE_SYSTEM_PROCESSOR = aarch64
+--> CMAKE_C_COMPILER = aarch64-linux-gnu-gcc
+--> CMAKE_SYSROOT = ${CMAKE_CURRENT_LIST_DIR}/rpi5-sysroot
+--> CMAKE_FIND_ROOT_PATH_MODE_LIBRARY = ONLY
+--> CMAKE_FIND_ROOT_PATH_MODE_INCLUDE = ONLY
+--> PKG_CONFIG_SYSROOT_DIR = ${CMAKE_SYSROOT}
```

Sysroot resolution: Defaults to rpi5-sysroot/ relative to the toolchain file. Override with -DSYSROOT_PATH=/custom/path.

8.2 Debug Level Configuration

CMakeLists.txt

```
|
+--> CMAKE_BUILD_TYPE = Debug
|   +--> DEBUG=1, DEBUG_LEVEL=4 (DEBUG)
|   +--> ENABLE_DEBUG_TRACE=ON -> DEBUG_LEVEL=5 (TRACE)
|
+--> CMAKE_BUILD_TYPE = Release
|   +--> DEBUG_LEVEL=2 (WARN only)
```

8.3 Source File Discovery

CMake uses GLOB_RECURSE to discover source files per module:

```
FILE(GLOB_RECURSE LVGL_Sources      CONFIGURE_DEPENDS lvgl/*.c)
FILE(GLOB_RECURSE UI_Sources        CONFIGURE_DEPENDS ui/*.c ui/*.cpp)
FILE(GLOB_RECURSE LOGIC_Sources     CONFIGURE_DEPENDS modules/logic/*.c)
FILE(GLOB_RECURSE UI_LOGIC_Sources  CONFIGURE_DEPENDS modules/ui_logic/*.c)
FILE(GLOB_RECURSE SOUND_LOGIC_Sources CONFIGURE_DEPENDS modules/sound_logic/*.c)
FILE(GLOB_RECURSE LED_LOGIC_Sources CONFIGURE_DEPENDS modules/led_logic/*.c)
FILE(GLOB_RECURSE GAME_MODE_Sources CONFIGURE_DEPENDS modules/game_modes/*.c)
FILE(GLOB_RECURSE DEBUG_Sources     CONFIGURE_DEPENDS modules/debug/*.c)
```

CONFIGURE_DEPENDS ensures CMake re-runs when new source files are added.

8.4 Link Dependencies

Squareline_Project (executable)

```
|
+--> SDL2          (display, input)
+--> SDL2_mixer    (audio)
+--> pio           (LED PIO control, from utils/piolib/)
+--> libgpiod      (GPIO sensor access)
+--> m             (math library)
```

9 Appendix A: Sensor Pin Mapping

Pin	Score	Sound File	LED Color	Debounce
GPIO 17	3 points	threePoints.wav	Green	3000 ms
GPIO 26	4 points	fourPoints.wav	Green	3000 ms
GPIO 27	5 points	fivePoints.wav	Green	3000 ms
GPIO 24	0 points	zeroPoints.wav	White	3000 ms

10 Appendix B: Sound File Inventory

10.1 Navigation Sounds

File	Trigger
backBtn.wav	Back button press
mainMenu.wav	Main menu button press
home.wav	Home navigation
scoreCard.wav	Scorecard view

10.2 Mode Selection Sounds

File	Trigger
strokePlay.wav	Stroke Play mode selected
matchPlay.wav	Match Play mode selected
quotaPoints.wav	Quota Points mode selected
vegasQuotaPoints.wav	Vegas Quota selected
onePlayer.wav - fourPlayers.wav	Player count selected
nineHoles.wav, eighteenHoles.wav	Hole count selected
oneVOne.wav, twoVTwo.wav	Match Play mode selected

10.3 Scoring Sounds

File	Trigger
threePoints.wav	3-point hole detected
fourPoints.wav	4-point hole detected
fivePoints.wav	5-point hole detected
zeroPoints.wav	0-point hole detected

10.4 Turn Sounds

File	Trigger
playerOne.wav - playerFour.wav	Player turn announcement
teamOne.wav, teamTwo.wav	Team turn announcement

10.5 Winner Sounds (5 variants each)

Pattern	Trigger
player1wins1.wav - player1wins5.wav	Player 1 wins (random pick)
player2wins1.wav - player2wins5.wav	Player 2 wins (random pick)
player3wins1.wav - player3wins5.wav	Player 3 wins (random pick)
player4wins1.wav - player4wins5.wav	Player 4 wins (random pick)
team1wins1.wav - team1wins5.wav	Team 1 wins (random pick)
team2wins1.wav - team2wins5.wav	Team 2 wins (random pick)

11 Appendix C: LVGL Configuration Summary

Parameter	Value	Notes
LVGL Version	9.1.0	
Color Depth	32-bit (XRGB8888)	
Memory Pool	512 MB	
Image Cache	8 MB	
Draw Layer Buffer	16 MB	
Refresh Period	16 ms (62.5 Hz)	
DPI	130	
Draw Units	4 (parallel SW)	Multi-core
Render Mode	Direct	Zero-copy to SDL
OS Support	None (bare loop)	
Default Font	Montserrat 14	
Text Encoding	UTF-8	
Performance Monitor	Enabled	FPS/CPU overlay
Memory Monitor	Enabled	Memory usage overlay
Image Decoders	PNG, BMP, JPEG, GIF	